



Chapter 19: Distributed Databases

Database System Concepts, 6th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Chapter 19: Distributed Databases

- ❑ Heterogeneous and Homogeneous Databases
- ❑ Distributed Data Storage
- ❑ Distributed Transactions
- ❑ Commit Protocols
- ❑ Concurrency Control in Distributed Databases
- ❑ Availability
- ❑ Distributed Query Processing
- ❑ Heterogeneous Distributed Databases
- ❑ Directory Systems



Distributed Database System

- A distributed database system consists of **loosely coupled sites** that share **no physical component**
- Database systems that run on each site are **independent** of each other
- Transactions may access data at **one or more sites**



Homogeneous Distributed Databases

- In a homogeneous distributed database
 - All sites have **identical software**
 - Are aware of each other and **agree to cooperate** in processing user requests.
 - Each site **surrenders part of its autonomy** in terms of right to change schemas or software
 - Appears to user as a **single system**
- In a heterogeneous distributed database
 - Different sites may use **different schemas and software**
 - ▶ Difference in schema is a **major problem** for query processing
 - ▶ Difference in software is a major problem for **transaction processing**
 - Sites may not be aware of each other and may provide only **limited facilities** for cooperation in transaction processing



Distributed Data Storage

- Assume relational data model
- Replication
 - System maintains **multiple copies** of data, stored in different sites, for faster retrieval and fault tolerance.
- Fragmentation
 - Relation is partitioned into **several fragments** stored in distinct sites
- Replication and fragmentation can be **combined**
 - Relation is partitioned into several fragments: system maintains several identical replicas of each such fragment.



Data Replication

- A relation or fragment of a relation is **replicated** if it is stored redundantly in two or more sites.
- **Full replication** of a relation is the case where the relation is stored at all sites.
- Fully redundant databases are those in which every site contains a copy of the entire database.



Data Replication (Cont.)

- Advantages of Replication
 - **Availability:** failure of site containing relation r does not result in unavailability of r if replicas exist.
 - **Parallelism:** queries on r may be processed by several nodes in parallel.
 - **Reduced data transfer:** relation r is available locally at each site containing a replica of r .
- Disadvantages of Replication
 - Increased cost of updates: each replica of relation r must be updated.
 - Increased complexity of concurrency control: concurrent updates to distinct replicas may lead to inconsistent data unless special concurrency control mechanisms are implemented.
 - ▶ One solution: choose one copy as **primary copy** and apply concurrency control operations on primary copy



Data Fragmentation

- Division of relation r into fragments $r_1, r_2, \dots, r_n$ which contain sufficient information to reconstruct relation r .
- **Horizontal fragmentation**: each tuple of r is assigned to one or more fragments
- **Vertical fragmentation**: the schema for relation r is split into several smaller schemas
 - All schemas must contain a common candidate key (or superkey) to ensure lossless join property.
 - A special attribute, the tuple-id attribute may be added to each schema to serve as a candidate key.



Horizontal Fragmentation of *account* Relation

<i>branch_name</i>	<i>account_number</i>	<i>balance</i>
Hillside	A-305	500
Hillside	A-226	336
Hillside	A-155	62

$$account_1 = \sigma_{branch_name="Hillside"}(account)$$

<i>branch_name</i>	<i>account_number</i>	<i>balance</i>
Valleyview	A-177	205
Valleyview	A-402	10000
Valleyview	A-408	1123
Valleyview	A-639	750

$$account_2 = \sigma_{branch_name="Valleyview"}(account)$$



Vertical Fragmentation of *employee_info* Relation

<i>branch_name</i>	<i>customer_name</i>	<i>tuple_id</i>
Hillside	Lowman	1
Hillside	Camp	2
Valleyview	Camp	3
Valleyview	Kahn	4
Hillside	Kahn	5
Valleyview	Kahn	6
Valleyview	Green	7

$deposit_1 = \Pi_{branch_name, customer_name, tuple_id}(employee_info)$

<i>account_number</i>	<i>balance</i>	<i>tuple_id</i>
A-305	500	1
A-226	336	2
A-177	205	3
A-402	10000	4
A-155	62	5
A-408	1123	6
A-639	750	7

$deposit_2 = \Pi_{account_number, balance, tuple_id}(employee_info)$



Advantages of Fragmentation

- Horizontal:
 - allows parallel processing on fragments of a relation
 - allows a relation to be split so that tuples are located where they are most frequently accessed
- Vertical:
 - allows tuples to be split so that each part of the tuple is stored where it is most frequently accessed
 - tuple-id attribute allows efficient joining of vertical fragments
 - allows parallel processing on a relation
- Vertical and horizontal fragmentation can be mixed.
 - Fragments may be successively fragmented to an arbitrary depth.



Data Transparency

- **Data transparency:** Degree to which system user may **remain unaware** of the details of how and where the data items are stored in a distributed system
- Consider transparency issues in relation to:
 - Fragmentation transparency
 - Replication transparency
 - Location transparency



Naming of Data Items - Criteria

1. Every data item must have a **system-wide unique name**.
2. It should be possible to **find the location of data items efficiently**.
3. It should be possible to **change the location of data items transparently**.
4. Each site should be able to create **new data items autonomously**.



Centralized Scheme - Name Server

- Structure:
 - name server assigns all names
 - each site maintains a record of local data items
 - sites ask name server to locate non-local data items
- Advantages:
 - satisfies naming criteria 1-3
- Disadvantages:
 - does not satisfy naming criterion 4
 - name server is a potential performance bottleneck
 - name server is a single point of failure



Use of Aliases

- Alternative to centralized scheme: each site prefixes its own site identifier to any name that it generates i.e., *site 17.account*.
 - Fulfills having a unique identifier, and avoids problems associated with central control.
 - However, fails to achieve network transparency.
- Solution: Create a set of **aliases** for data items; Store the mapping of aliases to the real names at each site.
- The user can be unaware of the physical location of a data item, and is unaffected if the data item is moved from one site to another.



Distributed Query Processing



Distributed Query Processing

- For centralized systems, the primary criterion for measuring the cost of a particular strategy is the **number of disk accesses**.
- In a distributed system, other issues must be taken into account:
 - The cost of a **data transmission over the network**.
 - The potential gain in performance from having **several sites** process parts of the query in parallel.



Query Transformation

- Translating algebraic **queries on fragments**.
 - It must be possible to **construct relation r** from its fragments
 - Replace relation r by the **expression to construct relation r** from its fragments

- Consider the horizontal fragmentation of the *account* relation into

$$account_1 = \sigma_{branch_name = \text{"Hillside"}}(account)$$

$$account_2 = \sigma_{branch_name = \text{"Valleyview"}}(account)$$

- The query $\sigma_{branch_name = \text{"Hillside"}}(account)$ becomes

$$\sigma_{branch_name = \text{"Hillside"}}(account_1 \cup account_2)$$

which is optimized into

$$\sigma_{branch_name = \text{"Hillside"}}(account_1) \cup \sigma_{branch_name = \text{"Hillside"}}(account_2)$$



Example Query (Cont.)

- Since $account_1$ has only tuples pertaining to the Hillside branch, we can eliminate the selection operation.
- Apply the definition of $account_2$ to obtain
$$\sigma_{branch_name = \text{"Hillside"}} (\sigma_{branch_name = \text{"Valleyview"}} (account))$$
- This expression is the empty set regardless of the contents of the $account$ relation.
- Final strategy is for the Hillside site to return $account_1$ as the result of the query.



Simple Join Processing

- Consider the following relational algebra expression in which the three relations are neither replicated nor fragmented

account ⋈ *depositor* ⋈ *branch*

- *account* is stored at site S_1
- *depositor* at S_2
- *branch* at S_3
- For a query issued at site S_1 , the system needs to produce the result at site S_1



Possible Query Processing Strategies

- Ship copies of all three relations to site S_1 and choose a strategy for processing the entire locally at site S_1 .
- Ship a copy of the account relation to site S_2 and compute $temp_1 = account \bowtie depositor$ at S_2 . Ship $temp_1$ from S_2 to S_3 , and compute $temp_2 = temp_1 \bowtie branch$ at S_3 . Ship the result $temp_2$ to S_1 .
- Devise similar strategies, exchanging the roles S_1, S_2, S_3
- Must consider following factors:
 - amount of data being shipped
 - cost of transmitting a data block between sites
 - relative processing speed at each site



Semijoin Strategy

- Let r_1 be a relation with schema R_1 stores at site S_1
Let r_2 be a relation with schema R_2 stores at site S_2
- Evaluate the expression $r_1 \bowtie r_2$ and obtain the result at S_1 .
 1. Compute $temp_1 \leftarrow \Pi_{R_1 \cap R_2}(r_1)$ at S_1 .
 - 2. Ship $temp_1$ from S_1 to S_2 .
 - 3. Compute $temp_2 \leftarrow r_2 \bowtie temp_1$ at S_2
 - 4. Ship $temp_2$ from S_2 to S_1 .
 - 5. Compute $r_1 \bowtie temp_2$ at S_1 . This is the same as $r_1 \bowtie r_2$.



Formal Definition

- The **semijoin** of r_1 with r_2 , is denoted by:

$$r_1 \bowtie r_2$$

- it is defined by:

$$\Pi_{R_1} (r_1 \Join r_2)$$

- Thus, $r_1 \bowtie r_2$ selects those tuples of r_1 that contributed to $r_1 \Join r_2$.
- In step 3 above, $temp_2 = r_2 \bowtie r_1$.
- For joins of several relations, the above strategy can be extended to a series of semijoin steps.



Join Strategies that Exploit Parallelism

- Consider $r_1 \bowtie r_2 \bowtie r_3 \bowtie r_4$ where relation r_i is stored at site S_i . The result must be presented at site S_1 .
- r_1 is shipped to S_2 and $r_1 \bowtie r_2$ is computed at S_2 : simultaneously r_3 is shipped to S_4 and $r_3 \bowtie r_4$ is computed at S_4
- S_2 ships tuples of $(r_1 \bowtie r_2)$ to S_1 as they produced;
 S_4 ships tuples of $(r_3 \bowtie r_4)$ to S_1
- Once tuples of $(r_1 \bowtie r_2)$ and $(r_3 \bowtie r_4)$ arrive at S_1 $(r_1 \bowtie r_2) \bowtie (r_3 \bowtie r_4)$ is computed in parallel with the computation of $(r_1 \bowtie r_2)$ at S_2 and the computation of $(r_3 \bowtie r_4)$ at S_4 .